# Relazione Progetto di Laboratorio di Algoritmi

Alex Di Corrado

6 Novembre 2026

## Indice

**1 Introduzione e Specifiche del Problema** **2**

**2 Struttura Dati Utilizzata e Implementazione** **2**
2.1 Motivazione della scelta . . . . . 2
2.2 Alfabeto Supportato . . . . . 2
2.3 La Classe TrieNode . . . . . 3
2.4 La Classe Trie . . . . . 3

**3 Descrizione della Soluzione Algoritmica** **3**
3.1 Inserimento di una Parola (insert) . . . . . 3
3.2 Ricerca per Prefisso e Visita DFS (searchByPrefix) . . . . . 4
3.3 Gestione dell'Input/Output (Classe Main) . . . . . 4

**4 Analisi della Complessità Computazionale** **5**
4.1 Complessità Temporale . . . . . 5
4.2 Complessità Spaziale . . . . . 6

**5 Verifica Sperimentale e Guida all'Uso** **6**
5.1 Suite di Test di Unità (Classe TrieTest) . . . . . 6
5.2 Istruzioni per la Compilazione e l'Esecuzione . . . . . 7

# 1 Introduzione e Specifiche del Problema

Il progetto richiede la realizzazione in Java di un motore di completamento automatico efficiente basato sulla struttura dati *Trie*, rispettando i seguenti requisiti:

1. inserire un insieme iniziale di parole all'interno del Trie;
2. ricevere un prefisso e restituire le parole corrispondenti;
3. ordinare alfabeticamente i suggerimenti restituiti;
4. consentire l'inserimento di nuove parole;
5. gestire ricerche prive di risultati e parole duplicate.

Inoltre, le specifiche impongono che il sistema non compia alcuna scansione sequenziale del dizionario per individuare i suggerimenti, dichiari esplicitamente l'insieme di caratteri supportato e gestisca correttamente il caso limite in cui il prefisso cercato coincida con una parola già memorizzata.

# 2 Struttura Dati Utilizzata e Implementazione

Per realizzare il sistema di completamento automatico è stato scelto il **Trie standard**. In questa struttura dati ogni nodo rappresenta un prefisso ed è implementato tramite un array di puntatori di dimensione fissa $|\Sigma| = 26$, in cui la contiguità dei codici ASCII permette di mappare direttamente ciascun indice alla corrispondente lettera dell'alfabeto $\Sigma = \{\text{'a'}, \ldots, \text{'z'}\}$, a partire dal carattere `'a'` all'indice 0 fino alla `'z'` all'indice 25. Le parole memorizzate si ottengono concatenando i caratteri lungo il cammino dalla radice ai nodi, mentre un flag booleano (`isRepresentative`) marca il nodo finale di ogni parola inserita per distinguerla dai prefissi intermedi (ad esempio `cas` rispetto a `casa` o `casale`).

## 2.1 Motivazione della scelta

Il Trie standard è stato preferito rispetto alle altre varianti per ragioni di efficienza e semplicità legate alle specifiche del problema:

- Rispetto al **Patricia Tree** (*Compact Trie*): pur comprimendo i cammini unari a $O(|R|)$ nodi, il Patricia Tree richiede di mantenere le stringhe in una struttura ausiliaria e rende complessa la gestione degli inserimenti dinamici (`INSERT`), poiché ogni nuova parola richiede di spezzare archi esistenti e aggiornare i riferimenti alle sottostringhe. Il Trie standard gestisce invece gli inserimenti in modo incrementale, allocando nuovi nodi unicamente dal punto di biforcazione.
- Rispetto al **Ternary Search Trie** (*TST*): il TST impiega tre soli puntatori per nodo ed è vantaggioso su alfabeti estesi, ma la sua altezza dipende dall'ordine di inserimento e può degradare a $O(n \cdot |S|)$ nel caso pessimo se non bilanciato. Con un alfabeto compatto di sole 26 lettere, il Trie standard garantisce transizioni $O(1)$ per carattere senza alcun bisogno di ribilanciamento e assicura l'ordinamento alfabetico nativo.

## 2.2 Alfabeto Supportato

L'alfabeto $\Sigma$ è dichiarato nella classe `TrieNode` e corrisponde alle ventisei lettere minuscole: $\Sigma = \{\text{'a'}, \text{'b'}, \ldots, \text{'z'}\}$, con $|\Sigma| = 26$. La corrispondenza bidirezionale tra caratteri e indici dell'array dei figli è gestita in tempo $O(1)$ da due funzioni statiche: `charToIndex(char c)` calcola la differenza numerica $c - \text{'a'}$, restituendo un indice nell'intervallo $[0, 25]$; `indexToChar(int index)`

calcola la somma 'a' + *index*, riconvertendo l'indice nel corrispondente carattere. Prima di procedere all'inserimento o alla ricerca, il metodo `isValidChar` convalida l'appartenenza di ciascun carattere a $\Sigma$.

### 2.3 La Classe TrieNode

Ogni istanza della classe `TrieNode` definisce un nodo dell'albero ed è composta da due soli campi:

- un array di riferimenti a nodi figli, denominato `children`, di dimensione fissa $|\Sigma| = 26$, in cui il valore `null` indica l'assenza di un arco uscente ($\perp$);
- un valore booleano `isRepresentative`, che indica se il nodo rappresenta la terminazione di una parola.

Il nodo non memorizza esplicitamente la lettera al suo interno, poiché il carattere associato a un arco è implicito nella posizione dell'indice all'interno dell'array del nodo padre. Questa scelta permette di ridurre l'occupazione di memoria.

### 2.4 La Classe Trie

La classe `Trie` incapsula l'intero albero mantenendo il riferimento al nodo radice (`root`), che rappresenta concettualmente la stringa vuota $\varepsilon$. La classe fornisce i metodi per la manipolazione della struttura: le operazioni primitive $O(1)$ `child(v, c)` e `setChild(v, c, u)` per la navigazione e il collegamento dei nodi, le procedure ad alto livello `insert(S)` e `searchByPrefix(prefix)` per l'inserimento e l'autocompletamento, e il metodo ausiliario `countNodes()` per il conteggio ricorsivo dei nodi.

## 3 Descrizione della Soluzione Algoritmica

L'applicazione articola le proprie funzionalità in tre algoritmi principali: l'inserimento, la ricerca del prefisso con relativa estrazione ordinata dei risultati e la gestione del flusso di input/output.

### 3.1 Inserimento di una Parola (insert)

Il metodo `insert(String S)` inserisce una parola di lunghezza $m$ nel Trie operando in tre passi:

1. **Convalida della stringa**: la stringa viene convertita in caratteri minuscoli rimuovendo eventuali spazi bianchi all'inizio e alla fine. Il metodo verifica poi che ogni carattere appartenga a $\Sigma$, sollevando un'eccezione se la parola risulta vuota o contiene caratteri non ammessi.
2. **Riuso del prefisso comune**: partendo dalla radice con un indice $j = 0$, un primo ciclo `while` fa avanzare il puntatore corrente $v$ lungo i rami già presenti finché ci sono ancora caratteri da esaminare ($j < m$) ed esiste già un arco uscente etichettato con la lettera corrente (`child(v, S[j]) != null`). Questa fase riutilizza il cammino comune più lungo senza allocare nuova memoria, arrestandosi sul nodo in cui termina il prefisso condiviso con le altre parole.
3. **Allocazione del suffisso**: a partire dal nodo raggiunto nel punto 2, un secondo ciclo `while` crea i rami mancanti finché ci sono ancora caratteri da inserire ($j < m$). In particolare a ogni passo alloca un nuovo `TrieNode` $u$, lo collega come figlio di $v$ lungo l'arco etichettato con la lettera corrente (`setChild(v, S[j], u)`) e fa avanzare il puntatore $v$ sul nuovo nodo $u$. Infine, una volta esaurita la parola, sul nodo terminale viene invocato `markAsRepresentative()`, impostando il relativo flag booleano a `true` per indicare che il cammino percorso individua una parola completa valida dell'insieme $R$.

Se si prova ad inserire una parola già presente, il primo ciclo scorre l'intera parola, il secondo non alloca alcun nodo e l'istruzione finale si limita a confermare lo stato rappresentativo. In questo modo si garantisce la corretta gestione dei duplicati senza spreco di memoria.

### 3.2 Ricerca per Prefisso e Visita DFS (searchByPrefix)

L'operazione di completamento automatico non effettua alcuna scansione sequenziale del dizionario, ma individua tutti i suggerimenti in due fasi:

1. **Discesa lungo il prefisso**: partendo dalla radice, un ciclo `for` scorre i caratteri del prefisso cercato accedendo a ogni passo al figlio corrispondente tramite `child(v, c)` in tempo costante $O(1)$. Se in qualunque momento si incontra un carattere non appartenente all'alfabeto $\Sigma$ o un arco nullo (`v == null`), il cammino si interrompe e la ricerca termina immediatamente restituendo una lista vuota. In questo modo si gestisce il caso di ricerche prive di riscontro senza mai scorrere il resto dell'albero. Se invece tutti i caratteri del prefisso sono presenti, il puntatore $v$ raggiunge la radice del sottoalbero contenente tutti e soli i vocaboli che iniziano con quel prefisso.

2. **Raccolta ordinata con visita DFS**: a partire dal nodo $v$ raggiunto nella prima fase, viene invocata la procedura ricorsiva `collectWordsDFS`, a cui vengono passati il nodo corrente, la lista in cui accumulare i suggerimenti e un buffer mutabile (`StringBuilder`) inizializzato con il prefisso cercato.

La procedura ricorsiva esplora il sottoalbero operando in pre-ordine. Per prima cosa, infatti, si verifica se il nodo corrente è marcato come rappresentativo: in caso affermativo (`isRepresentative == true`), la sequenza di caratteri accumulata nel buffer individua una parola valida e viene immediatamente inserita nella lista dei suggerimenti.

Successivamente un ciclo `for` esamina in ordine crescente tutti gli indici da 0 a 25. Per ogni arco esistente (`nextNode != null`), il metodo converte l'indice nel corrispondente carattere tramite `indexToChar`, lo aggiunge in coda al buffer (`append`) e richiama se stesso ricorsivamente sul nodo figlio. Al ritorno dalla chiamata ricorsiva, l'ultimo carattere inserito viene rimosso dal buffer (`deleteCharAt`).

In questo modo l'algoritmo risolve per costruzione due requisiti della traccia, ovvero l'ordinamento alfabetico dei suggerimenti e il caso in cui il prefisso coincide con una parola completa. Infatti, poiché gli archi vengono esaminati nell'ordine alfabetico dei loro indici (dalla `'a'` alla `'z'`), la visita in profondità estrae le parole già ordinate lessicograficamente, senza richiedere una successiva operazione di ordinamento.

Inoltre, nel caso in cui il prefisso digitato coincida esattamente con una parola memorizzata nel vocabolario (come `casa` a fronte delle parole `casa` e `casale`), il nodo di partenza della visita ha già `isRepresentative == true`: la procedura include quindi tale parola come primo suggerimento della lista prima di scendere ad esplorare le parole più lunghe.

### 3.3 Gestione dell'Input/Output (Classe Main)

La classe `Main` gestisce il flusso I/O. Il metodo `processInput` legge la prima riga per ottenere il numero intero $N$ di parole iniziali, inserisce le successive $N$ parole nel Trie e avvia un ciclo di ascolto continuo. Per ogni comando successivo: se riceve `PREFIX <prefisso>`, interroga il Trie e stampa i risultati, oppure la stringa `(nessun suggerimento)` se la lista è vuota; se riceve `INSERT <parola>`, aggiunge il termine al Trie ed emette la riga di conferma `Parola inserita: <parola>`. Il programma supporta sia l'esecuzione da file tramite argomento da terminale (`FileInputStream`), sia la modalità interattiva da tastiera (`System.in`).

# 4 Analisi della Complessità Computazionale

Per analizzare la complessità di tempo e di spazio della soluzione implementata, si considerano:

- l'alfabeto supportato $\Sigma = \{\text{'a'}, \ldots, \text{'z'}\}$ ($|\Sigma| = 26$);
- l'insieme $R$ delle $n$ parole memorizzate con lunghezza complessiva $||R|| = \sum_{S \in R} |S|$;
- il prefisso cercato $P$ di lunghezza $|P|$;
- il numero $k$ di risultati (suggerimenti) restituiti dalla ricerca per prefisso.

## 4.1 Complessità Temporale

**Inserimento (`insert`).** Ogni nodo del Trie è implementato con un array di dimensione fissa $|\Sigma| = 26$. Poiché la dimensione dell'alfabeto è costante, le operazioni di conversione del carattere in indice e di accesso o assegnazione del figlio (`child` e `setChild`) hanno costo costante $O(1)$. Inserire una parola $S$ di lunghezza $|S|$ consiste nel seguire il cammino dalla radice scandendo i caratteri di $S$ uno alla volta: finché i caratteri sono già presenti si scende lungo i nodi esistenti, mentre non appena si incontra un riferimento `null` si creano i nuovi nodi per i caratteri rimanenti. Poiché per ciascuno dei $|S|$ caratteri si svolge un numero costante di operazioni, con alfabeto di dimensione costante il costo per l'inserimento di una stringa $S$ è:

$$O(|S|)$$

Nel caso di inserimento di una parola duplicata (già presente nel Trie), il primo ciclo scorre i $|S|$ nodi esistenti senza allocare nuovi nodi, mantenendo il costo $O(|S|)$. Di conseguenza, la costruzione del Trie per l'intero insieme di parole $R$ ha un costo temporale proporzionale alla lunghezza complessiva delle parole memorizzate:

$$O(||R||)$$

**Ricerca per prefisso (`searchByPrefix`).** La ricerca di una stringa in un Trie con alfabeto costante ha costo lineare nella sua lunghezza. Nel caso del completamento automatico, l'operazione si divide in due fasi:

1. **Discesa lungo il prefisso:** partendo dalla radice, l'algoritmo segue il cammino corrispondente ai caratteri del prefisso $P$, effettuando a ogni passo un accesso `child(v, c)` in tempo $O(1)$. Questa fase richiede tempo $O(|P|)$ senza mai effettuare una scansione sequenziale del dizionario. Se il prefisso non è presente nel Trie (ricerca senza risultati, $k = 0$), il cammino si interrompe non appena si incontra un puntatore `null` e la ricerca termina immediatamente in tempo $O(|P|)$.
2. **Raccolta ordinata dei risultati (visita DFS):** se il prefisso è presente, dal nodo raggiunto parte la visita in profondità (`collectWordsDFS`) che esplora il sottoalbero del prefisso per raccogliere le $k$ parole corrispondenti. Per ciascun nodo visitato, il ciclo `for` controlla le $|\Sigma| = 26$ celle dell'array `children` in tempo costante $O(|\Sigma|) = O(1)$, aggiungendo alla lista dei suggerimenti le $k$ parole marcate come rappresentative.

Il tempo richiesto dalla seconda fase dipende dal numero di risultati $k$ e dalla lunghezza delle parole trovate nel sottoalbero. Nel caso pessimo, in cui tutte le parole memorizzate nel dizionario iniziano con il prefisso cercato $P$, la visita esplora tutti i nodi del Trie impiegando tempo $O(||R||)$, per un costo complessivo nel caso pessimo pari a:

$$O(|P| + ||R||)$$

Inoltre, poiché l'array dei figli è indicizzato alfabeticamente dalla `'a'` alla `'z'`, la visita restituisce i $k$ suggerimenti già ordinati lessicograficamente, senza dover eseguire alcun algoritmo di ordinamento aggiuntivo sui risultati.

### 4.2 Complessità Spaziale

**Numero di nodi nel Trie.** Il numero esatto di nodi presenti in $trie(R)$ è stabilito dal seguente teorema:

$$||R|| - \text{ELCP}(R) + 1$$

dove $||R||$ è la lunghezza totale delle stringhe in $R$ ed $\text{ELCP}(R)$ indica la somma delle lunghezze dei prefissi comuni condivisi tra le parole dell'insieme $R$. Inoltre, se $R$ è un insieme di $n$ stringhe, il Trie non può avere più di $n$ nodi di biforcazione (*branching*) e possiede $O(||R||)$ nodi unari:

- Nel caso pessimo, in cui le parole memorizzate non condividono alcun prefisso comune ($\text{ELCP}(R) = 0$), il numero di nodi è massimo ed è pari a $||R|| + 1$, quindi $O(||R||)$.
- Quando invece le parole condividono prefissi comuni ($\text{ELCP}(R) > 0$), i nodi dei prefissi vengono riutilizzati riducendo la memoria allocata; in particolare, l'inserimento di parole duplicate non crea alcun nuovo nodo.

**Spazio complessivo occupato.** Poiché ogni nodo del Trie è implementato mediante un array di dimensione fissa $|\Sigma| = 26$ più una variabile booleana, e il numero totale di nodi è $O(||R||)$, lo spazio occupato dalla struttura dati è:

$$O(||R|| \cdot |\Sigma|) = O(||R||)$$

Durante l'esecuzione della ricerca per prefisso `searchByPrefix`, la memoria ausiliaria utilizzata è data dallo `StringBuilder` e dallo stack delle chiamate ricorsive della visita DFS (proporzionali alla lunghezza della parola più lunga), oltre alla lista restituita in output che contiene le $k$ parole trovate (di dimensione nulla se $k = 0$, e al più $O(||R||)$ se vengono restituite tutte le parole del dizionario).

**Sintesi rispetto ai parametri del problema.** Riassumendo in funzione della lunghezza del prefisso $|P|$, del numero di risultati $k$ e della lunghezza complessiva delle parole memorizzate $||R||$:

- **Complessità Temporale:** L'inserimento di tutte le parole nel Trie richiede tempo $O(||R||)$ (e $O(|S|)$ per ogni singola nuova parola $S$). La ricerca per prefisso richiede tempo $O(|P|)$ per individuare il sottoalbero (o per terminare se $k = 0$), a cui si somma il tempo di visita del sottoalbero per estrarre in ordine alfabetico i $k$ risultati, fino a un massimo di $O(|P| + ||R||)$ nel caso pessimo in cui tutte le parole memorizzate inizino con $P$.
- **Complessità Spaziale:** Il Trie occupa uno spazio pari a $O(||R|| \cdot |\Sigma|) = O(||R||)$ distribuito su esattamente $||R|| - \text{ELCP}(R) + 1$ nodi, mentre la ricerca per prefisso richiede uno spazio di output proporzionale alla lunghezza complessiva dei $k$ risultati restituiti (nullo se $k = 0$, e al più $O(||R||)$).

## 5 Verifica Sperimentale e Guida all'Uso

### 5.1 Suite di Test di Unità (Classe TrieTest)

La correttezza dell'algoritmo e il rispetto dei requisiti della traccia sono stati verificati tramite una suite di nove test automatizzati con **JUnit 5**, implementati nella classe `TrieTest`.

I primi test verificano il corretto funzionamento dei singoli nodi e la gestione dell'alfabeto. In particolare, `testNewNodeInitialization` e `testMarkAsRepresentative` controllano che ogni nuovo nodo nasca con tutti i riferimenti ai figli posti a `null` e non sia marcato come parola completa fino all'invocazione di `markAsRepresentative()`. Con `testAlphabetMappingAndValidation`

viene invece verificata la conversione bidirezionale tra caratteri minuscoli e indici numerici in tempo $O(1)$, accertando che caratteri non supportati (come lettere maiuscole, numeri o simboli) vengano rifiutati sollevando un'eccezione.

I test successivi si concentrano sul comportamento del Trie durante gli inserimenti e le ricerche, verificando anche tutti i casi limite previsti. Il metodo `testNodeCountTheorem` convalida la proprietà teorica sul numero totale di nodi ($||R|| - \text{ELCP}(R) + 1$): inserendo le sei parole d'esempio $R = \{\texttt{ali}, \texttt{alice}, \texttt{anna}, \texttt{eliana}, \texttt{elias}, \texttt{elisa}\}$, il conteggio tramite `countNodes()` restituisce esattamente $28 - 11 + 1 = 18$ nodi. La gestione dei duplicati viene collaudata da `testDuplicateWordsHandling`, che accerta come l'inserimento ripetuto di parole già presenti non crei nodi superflui né produca doppioni nell'output. I metodi `testPrefixIsCompleteWord` e `testAlphabeticalOrdering` verificano rispettivamente che un prefisso coincidente con una parola completa (come `casa` rispetto a `casale`) compaia tra i suggerimenti e che le parole vengano restituite sempre in ordine alfabetico, anche se inserite in ordine sparso o inverso. Infine, `testSearchWithNoResults` controlla che le ricerche prive di corrispondenza restituiscano immediatamente una lista vuota, mentre `testExamTraceExample` riproduce fedelmente l'intera sequenza di comandi riportata nella traccia d'esame.

## 5.2 Istruzioni per la Compilazione e l'Esecuzione

Per verificare il comportamento dell'interfaccia a riga di comando della classe `Main`, sono forniti due file di test nella cartella `input/`: `input1_traccia.txt` riproduce fedelmente l'esempio ufficiale della traccia d'esame per verificare il flusso base di esecuzione, mentre `input2_completo.txt` è strutturato per collaudare i casi limite del sistema, come prefissi coincidenti con parole complete (`ali` e `alice`), l'inserimento dinamico di parole duplicate (`INSERT ali`) e ricerche senza corrispondenze nel dizionario (`PREFIX zzz`).

L'applicazione può essere compilata ed eseguita da terminale tramite i seguenti comandi:

1. **Compilazione ed esecuzione dei test automatizzati:**

```
mvn clean test
```

2. **Esecuzione con i file di input:**

```
java -cp target/classes it.unipa.algoritmi.Main input/input1_traccia.txt
java -cp target/classes it.unipa.algoritmi.Main input/input2_completo.txt
```

3. **Esecuzione interattiva da console:**

```
java -cp target/classes it.unipa.algoritmi.Main
```